

UCS503 – Software Engineering Project

Academic Year 2026–2027

AI Cost Autopilot

*A Compiler-Aware Gateway for Token-Cost Optimisation
in LLM Coding Agents*

Submitted by:

1024240033	Agampreet Kaur	<i>Compiler-Aware Trimmer</i>
1024240012	Devansh Sharma	<i>Semantic Cache</i>
1024240029	Furmaandeeep Kaur	<i>Autopilot Policy</i>

Computer Science and Engineering Department (CSED)

Submitted to:

Dr. Jeelani Asif

Department of Computer Science and Engineering

Repository: <https://github.com/agamlotey/ucs503p-202627-ai-cost-autopilot>

Documentation: <https://agamlotey.github.io/ucs503p-202627-ai-cost-autopilot/>

Prototype Stage Report

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	1
1.1	Background and Context	1
1.1.1	Problem Statement	1
1.2	Project Objectives	1
1.3	Scope of this Prototype	2
2	Methodology and Design	3
2.1	System Architecture	3
2.1.1	High-Level Design	3
2.1.2	Design Diagrams	4
2.1.3	Technical Stack	7
2.2	Implementation Details	7
2.2.1	Gateway	7
2.2.2	Module 1: Compiler-Aware Trimmer	8
2.2.3	Module 2: Semantic Cache	8
2.2.4	Module 3: Autopilot	9
2.3	Development Process	9
3	Results and Evaluation	10
3.1	Testing Methodology	10
3.2	Performance Metrics	10
3.2.1	Trimmer against simple baselines	10
3.2.2	Cache: can similarity be trusted for code?	11
3.2.3	Cache: savings depend on repetition	11
3.2.4	Cache: added latency	11
3.2.5	Autopilot: secret detection	12
3.2.6	End-to-end demonstration	12
3.3	Analysis	12
4	Conclusion	13
4.1	Summary of Work	13
4.2	Key Findings	13
4.3	Limitations	13
4.4	Future Work and Recommendations	13
4.5	Final Remarks	14

Chapter 1

Introduction

1.1 Background and Context

AI coding agents such as Cursor, Claude Code and Codex are now part of everyday software development. Every time one of them answers a request, it sends a large slice of the codebase to a large language model (LLM), and the provider bills for every input and output token. As teams and codebases grow, this becomes one of the least predictable lines in an engineering budget. Much of what is sent is not needed for the task at hand: whole files are included when one function matters, and the same questions are asked again and again.

Before building anything of our own, we checked that the idea was worth pursuing. We routed the open-source TestZeus Hercules agent through a proxy built from *existing* tools (Headroom prompt compression behind a LiteLLM proxy). That proof of concept used about 25% fewer tokens and cost about 17% less per run, with no change in agent behaviour across repeated runs. These figures belong to the existing tools, not to our system; they only show that a gateway between an agent and its model can cut cost without breaking the agent.

The same pattern appears in industry. Databricks describes an AI Gateway that routes requests across models, compresses context and tunes prompt caching, and reports more than 30% lower average task cost from routing and almost 50% fewer generated tokens from harness and cache tuning, with no observed loss of quality [1]. Our project builds this kind of gateway ourselves, and tests whether a *language-aware* way of shrinking context does better than generic text compression.

1.1.1 Problem Statement

Coding agents send far more code than a task needs, and they repeat requests. Both waste money. Generic text compressors reduce tokens but know nothing about code structure, so they can break the code they compress. There is no standard layer that reduces this spend while keeping the request correct.

The problem we address is: **reduce the tokens a coding agent sends to an LLM, without removing anything the task needs and without producing invalid code**, using a gateway that the agent can use with no change to its workflow.

1.2 Project Objectives

1. Build a gateway that speaks the OpenAI-compatible `/v1/chat/completions` API, so an agent only has to change its base URL to use it.
2. Build a **compiler-aware trimmer** that parses the code in a request, keeps the functions the task is about together with the functions they call, and collapses the rest to signatures, always producing valid code.
3. Build a **semantic cache** that returns a stored answer when a new request has the same meaning as an earlier one, and never returns a wrong answer for code.
4. Build an **autopilot** policy that chooses the cheapest safe action for each request.
5. Measure the savings honestly against simple baselines, and target at least 30% token reduction on code-heavy, repetitive workloads.

1.3 Scope of this Prototype

Following the roadmap, the prototype stage delivers the gateway running end to end with a Python trimmer, an exact-match cache (with semantic matching for prose), and a basic autopilot. A mock provider lets the whole pipeline run offline with no API key and no cost. Later stages add a second language, a tuned autopilot, a metrics store and a benchmark across agents.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

The system is a single gateway placed between the coding agent and the LLM provider. Each request goes through the same pipeline (Figure 2.1):

1. **Snapshot.** The gateway keeps an unmodified copy of the request. The cache always uses this original copy, so a request that is later trimmed can still be found again.
2. **Signals.** It computes cheap signals: an estimated token count and whether the request contains code.
3. **Plan.** The autopilot decides whether to use the cache and whether to trim.
4. **Cache.** On a hit, the stored response is returned at once and the LLM is never called.
5. **Trim.** Otherwise, if the plan says so and the request is over the token budget, the trimmer reduces the code context.
6. **Forward and store.** The request is forwarded to the provider, and the response is stored in the cache under the original request before it is returned.

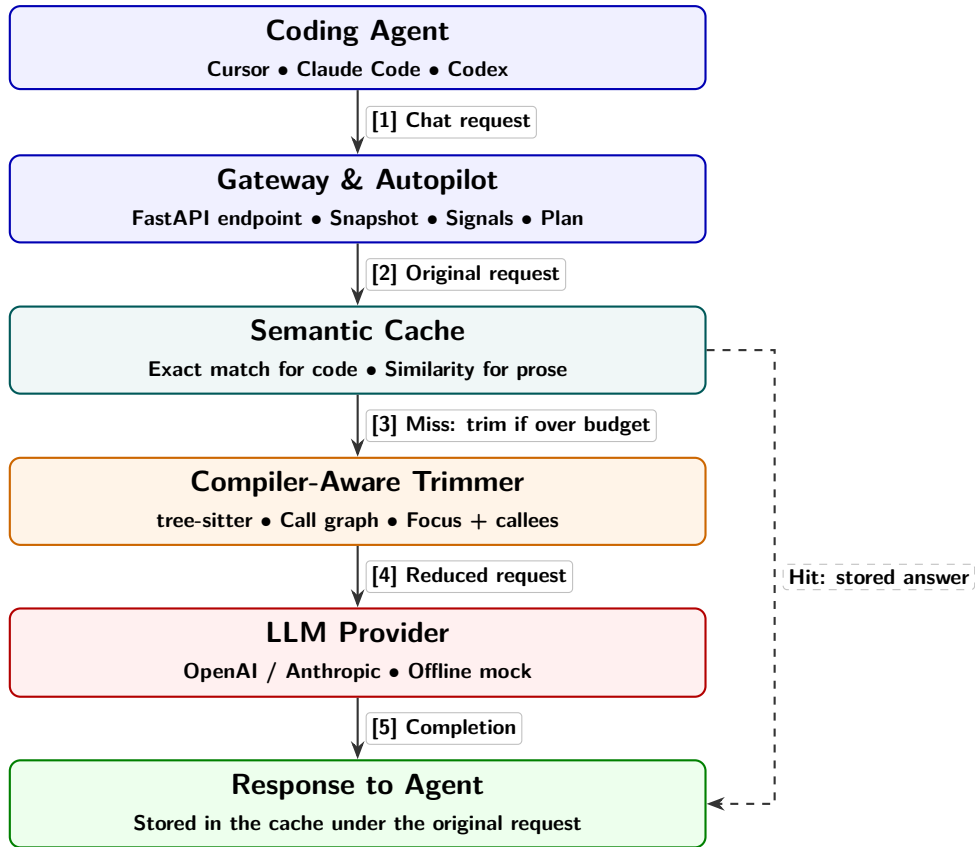


Figure 2.1: High-level request path through the gateway. On a cache hit (dashed) the stored answer is returned and the model is never called.

The three components are developed independently against interfaces frozen early in the project (`gateway/interfaces.py`), so each teammate can build and test their part with mocks. The gateway is the only place they meet.

2.1.2 Design Diagrams

The design is documented with four diagrams, all available on the project site and in `docs/Diagrams/`:

- **Use case diagram** (Figure 2.2): the Developer and the Coding Agent send requests; reducing cost is always included, while reusing an answer, reducing code context and calling the LLM are conditional extensions.
- **Data flow diagram** (Figure 2.3): Level 0 shows the gateway and its external entities, Level 1 its six processes and two data stores, and Level 2 the seven steps of the trimmer.
- **Activity (swimlane) diagram** (Figure 2.4): one request through the gateway, with one lane per component, decisions, and fork/join for independent work.
- **ER diagram** (Figure 2.5): the conceptual data model, covering cache entries, metric records, and the trimmer's model of source files, functions and the calls between them.

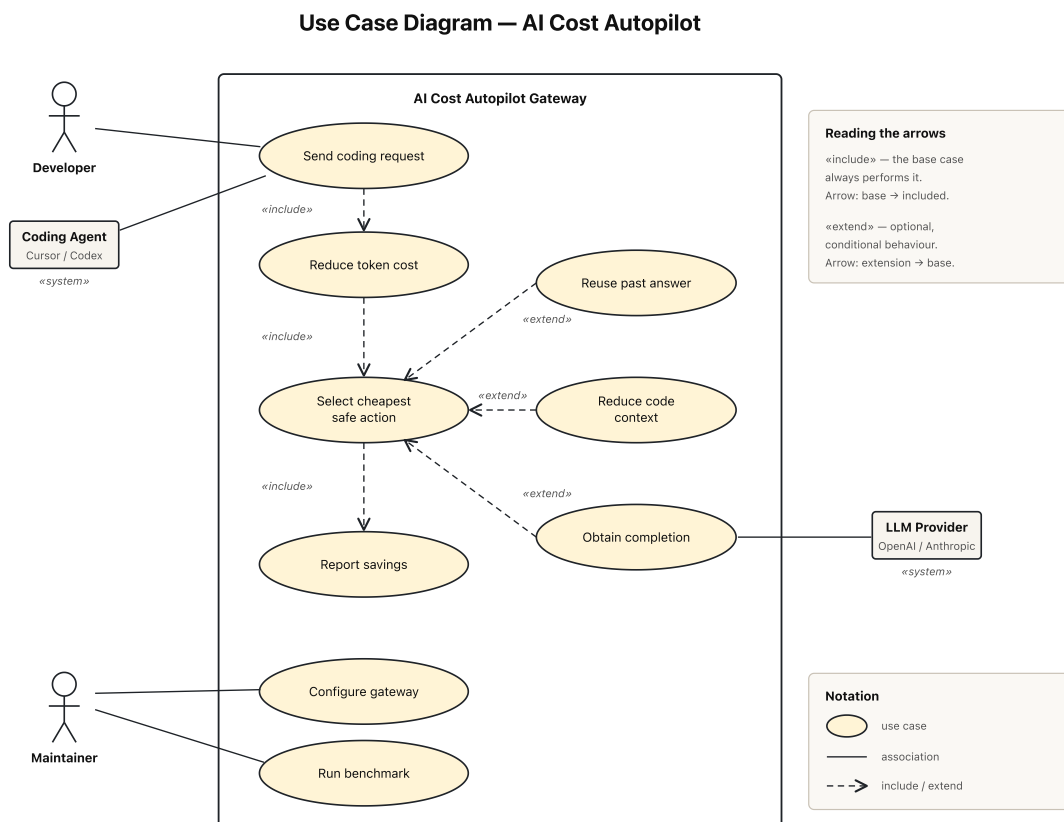


Figure 2.2: Use case diagram.

Data Flow Diagram — AI Cost Autopilot

Levels 0, 1 and 2 · Gane-Sarson notation

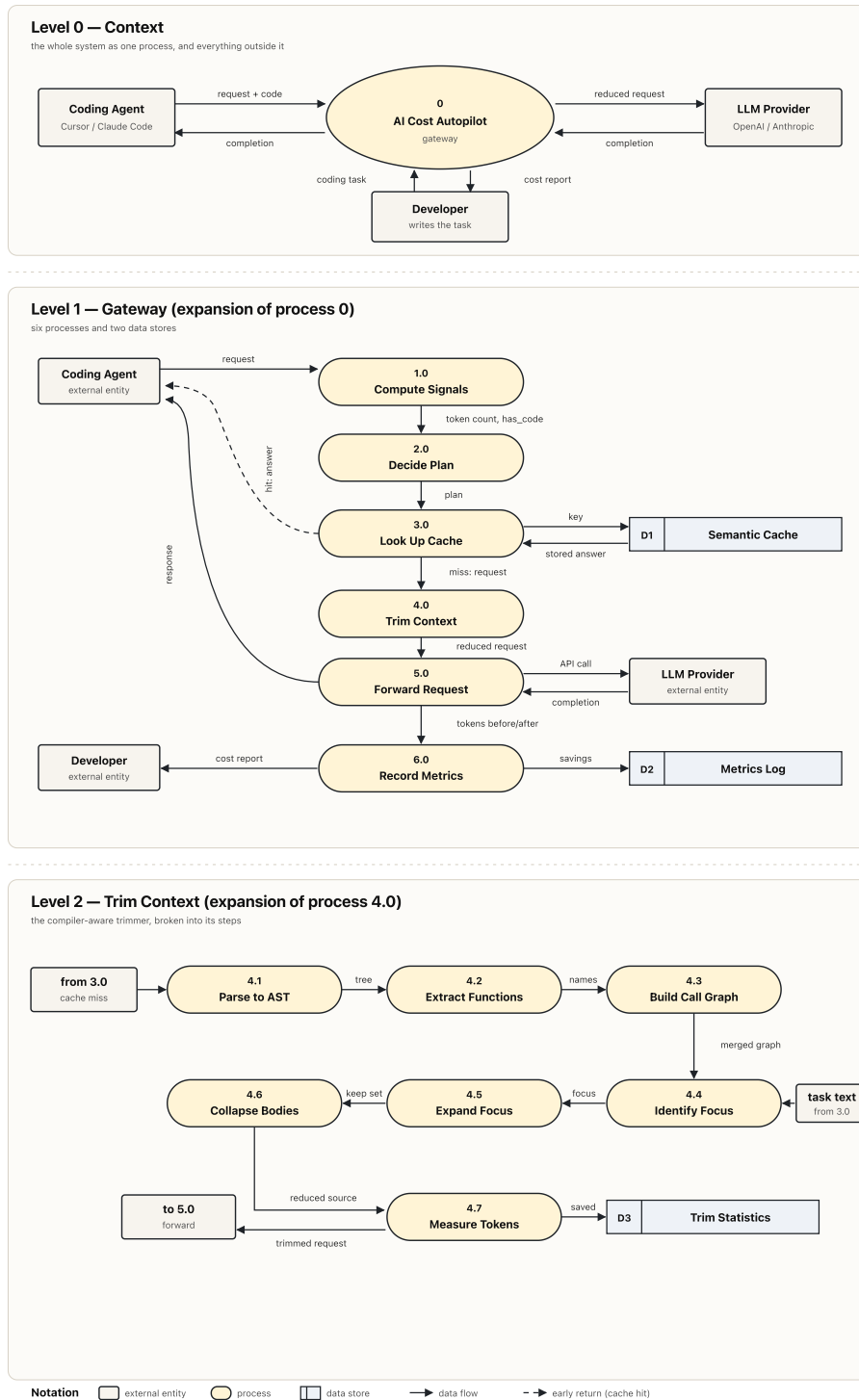


Figure 2.3: Data flow diagram, levels 0, 1 and 2.

Activity Diagram: Handling a Coding Request

Use case: Send coding request. Each swimlane shows which component performs the action.

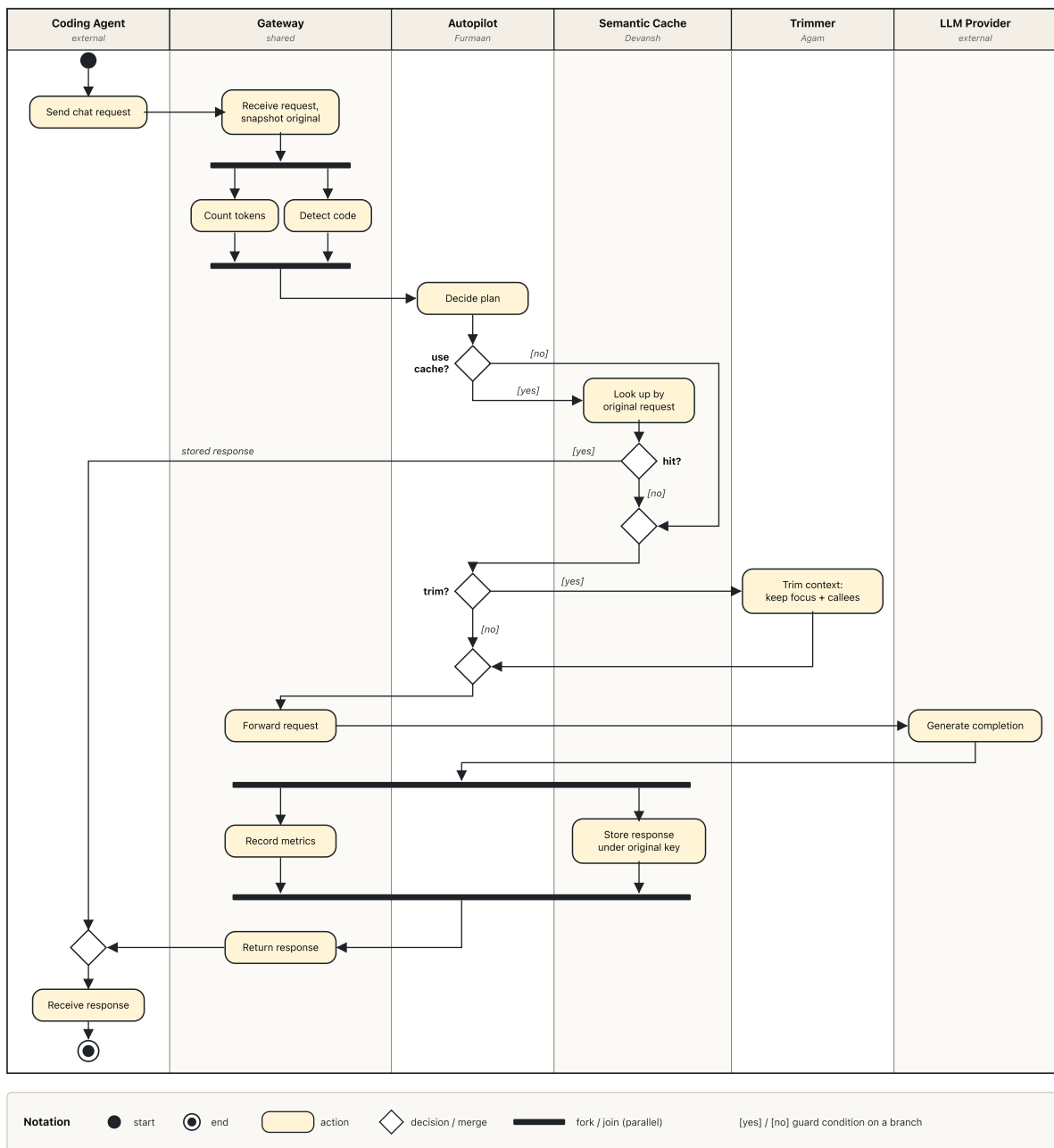


Figure 2.4: Activity diagram with swimlanes for handling one coding request.

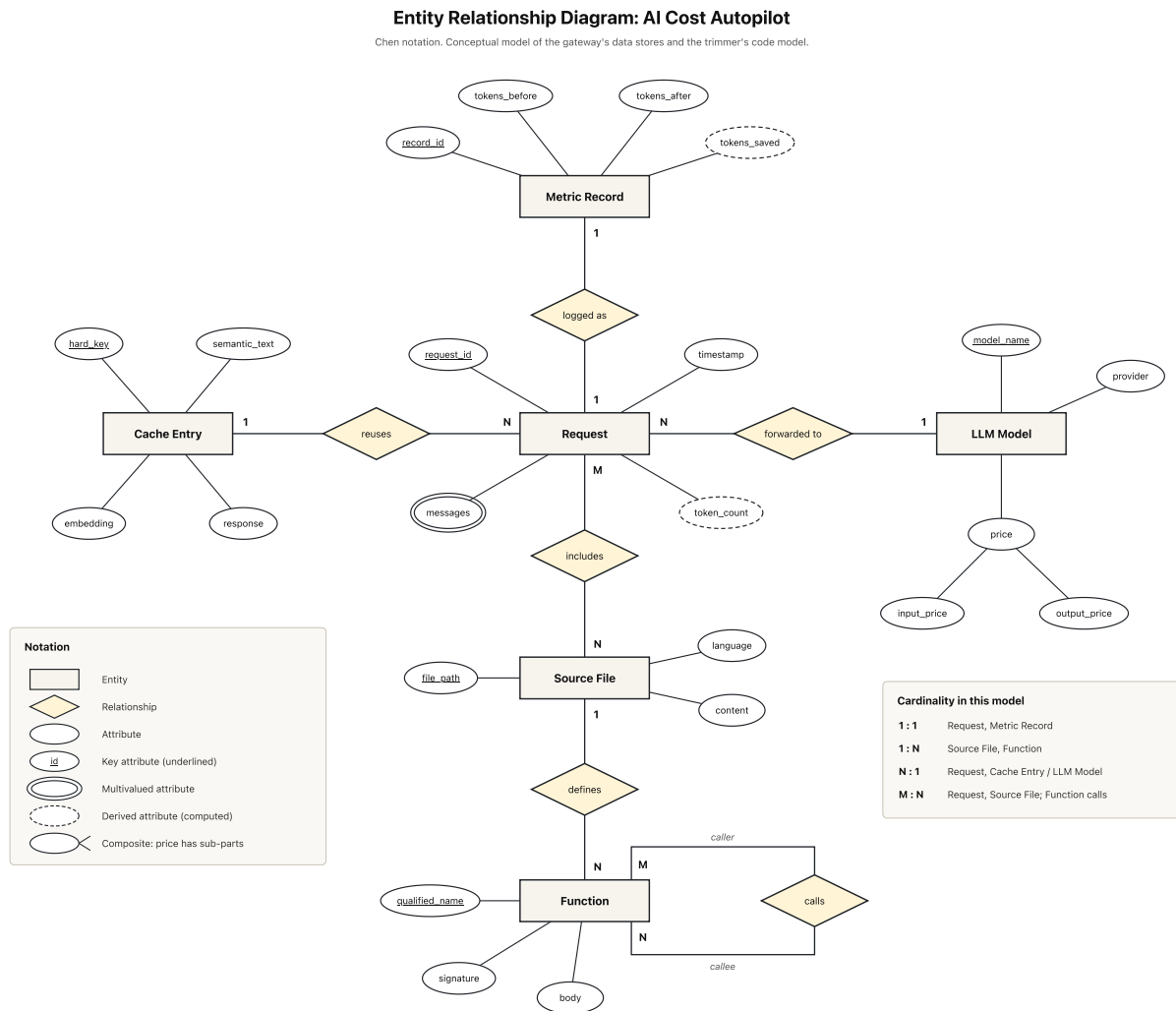


Figure 2.5: Entity relationship diagram (Chen notation).

2.1.3 Technical Stack

Layer	Technology
Language	Python 3.12 (CI) and 3.13 (development)
Gateway	FastAPI, Uvicorn, httpx for forwarding
Parsing	tree-sitter with the <code>tree-sitter-python</code> grammar
Token counting	tiktoken, <code>cl100k_base</code> encoding
Semantic matching	sentence-transformers, <code>all-MiniLM-L6-v2</code> (optional)
Testing and CI	pytest; GitHub Actions on every pull request and push to <code>master</code>
Documentation	MkDocs Material on GitHub Pages; LaTeX for reports

Table 2.1: Technical stack.

2.2 Implementation Details

2.2.1 Gateway

The gateway exposes `POST /v1/chat/completions` and `GET /health`. It takes a deep copy of the request before anything changes it, computes the signals, asks the autopilot for a plan,

and then runs the cache, trimmer and provider in that order. A **mock mode** returns a well-formed completion (with an id, timestamp and estimated token usage) instead of calling a real provider. It turns on automatically when no API key is set, so every teammate and the CI can run the full pipeline with no key and no cost.

An early bug showed why the snapshot matters: the cache originally stored responses *after* the trimmer had rewritten the request, so a response was filed under the trimmed text and an identical request could never hit it. Keying both lookup and store on the original request fixed this.

2.2.2 Module 1: Compiler-Aware Trimmer

The trimmer works on the syntax tree rather than on raw text.

1. **Parse.** Each code message is parsed with tree-sitter [2].
2. **Collapse.** A function that is not needed keeps its signature and docstring, and its body is replaced by an ellipsis (...). Edits are applied from the end of the file backwards using byte offsets, so earlier positions stay valid and indentation is preserved. The result is always valid Python.
3. **Call graph.** The trimmer records which functions each function calls. Calls through `super()` are ignored, because they go to the base class; treating them as calls to a same-named local function created false edges (for example `__init__` appearing to call itself). Self-edges from recursion are dropped.
4. **Focus.** The functions the task is about are taken from names mentioned in the plain-text part of the request (or passed explicitly).
5. **Expand.** A breadth-first search from the focus follows the call graph for up to two hops. Those functions are kept in full.

Cross-file dependencies. A request usually carries one file per message. The first version built a call graph per message, so a dependency defined in another file was missed. On a realistic fixture, asking to “fix `create_note`” kept `create_note` but collapsed `validate_note`, which lives in a different file and is exactly what the fix needs. The trimmer now merges one call graph across every code message in the request, resolving calls against all functions defined anywhere in it. Figure 2.6 shows the whole pipeline.

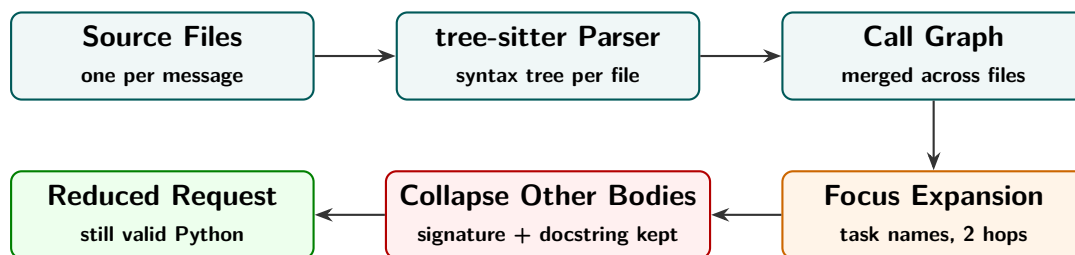


Figure 2.6: Compiler-aware trimming pipeline.

2.2.3 Module 2: Semantic Cache

Each cache entry is split into a **hard key** and the **message text**. The hard key is a hash of everything that must match exactly: the model, every parameter (temperature, maximum tokens, tools, response format and so on) and the message roles. Only an explicit list of fields known not to affect the answer is left out, so an unknown parameter makes the cache miss rather than return a wrong answer.

Within a hard key, lookup works in three steps:

1. **Exact text match**, which needs no embedding model and covers both code and prose.
2. **Code is exact-only**. If the request contains code, the cache never reuses on similarity (see Section 3.2.2).
3. **Prose uses meaning**. For natural language, the text is embedded with a sentence-embedding model [4] and the most similar stored entry is reused if its cosine similarity is at least 0.90.

Storing the same request again replaces the old entry instead of adding a duplicate. If sentence-transformers is not installed, exact matching still works and only similarity matching is disabled. Figure 2.7 shows the lookup flow.

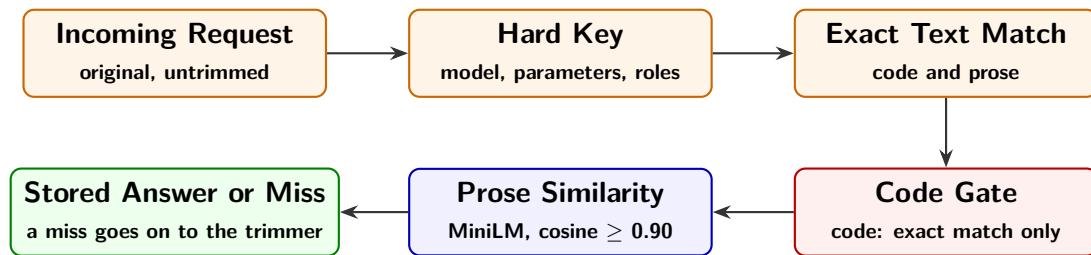


Figure 2.7: Semantic cache lookup flow.

2.2.4 Module 3: Autopilot

The autopilot turns the signals into a plan, in order of cost. First, a **safety check**: if the request contains a secret it is never cached, neither read from the cache nor written to it. The scan covers API keys in their current formats (OpenAI `sk-proj-`, Anthropic `sk-ant-`, Google, GitHub, AWS), private keys, tokens, and quoted password or key literals, and it reads multipart messages as well as plain text. It deliberately ignores ordinary code such as `password = payload.get("password")`, which would otherwise switch off caching for any code that handles logins. Second, the cache is always tried. Third, trimming is requested when the request contains code and is estimated at over 1000 tokens; a secret does not stop trimming, because trimming caches nothing and sends less code to the provider. The autopilot also counts its decisions, as the basis for tuning its thresholds later.

The trimmer applies its own 8000-token budget on top of the autopilot’s 1000-token threshold, so for requests in between the plan asks for trimming that does not happen. Aligning the two thresholds is next-stage work.

2.3 Development Process

Work happens on feature branches. Every change to `master` goes through a pull request that needs one approving review from a teammate, and the GitHub Actions test workflow runs on every pull request and on every push to `master`. Reviews have caught real defects in both directions: missing parameters in the cache key and a cache that returned the oldest duplicate were found in review, as was the trimmed-key bug described above. Each member keeps a weekly technical journal in the repository.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

The test suite has 65 tests: 24 for the trimmer, 21 for the cache, 13 for the autopilot and 7 for the gateway, most of which send real HTTP requests through the whole pipeline using the mock provider. At the time of writing, 64 pass and 1 is skipped (the test that needs the real embedding model, which is not installed in CI). Figure 3.1 shows how the suites feed continuous integration.

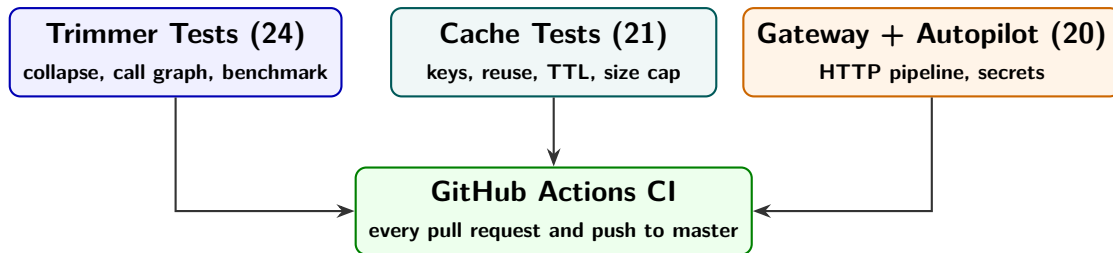


Figure 3.1: Test suites and continuous integration.

Measurements use a realistic fixture, `notes_api`: a small but genuine service with 10 files, 45 functions and 3,277 tokens, with real imports between files. A toy project of two files was not enough, because in a project that small almost every function is reachable from every other, so there is nothing to trim selectively. Every file in the fixture is checked to compile, and tokens are counted with `tiktoken` [3] (`cl100k_base`).

3.2 Performance Metrics

3.2.1 Trimmer against simple baselines

Table 3.1 compares the trimmer with simpler ways of shrinking the same request, for the task “fix `create_note`”. Every variant is measured on the same file contents, and “valid” means every output file still compiles as Python. The table is produced by `python -m trimmer.benchmark.baselines`, and CI checks the relationships it shows.

Variant	Tokens	Saved	Output
Raw request (no gateway)	3,277	–	valid
Strip indentation and blank lines	2,982	9.0%	broken
Strip comments and docstrings	2,606	20.5%	valid
Collapse every function body	1,410	57.0%	valid, but drops the focus
Trimmer (focus and callees, 2 hops)	2,166	33.9%	valid

Table 3.1: Token reduction on the `notes_api` fixture.

The trimmer kept 18 of the 45 functions in full, including `validate_note` from another file, and collapsed the other 27 to their signatures. “Fix `create_note`” is the hardest task on this fixture because it is the most connected function; on three other tasks the trimmer saves between 47% and 55%.

3.2.2 Cache: can similarity be trusted for code?

We scored labelled pairs of requests with the embedding model. A safe threshold needs every same-meaning pair above it and every different-meaning pair below it.

Group	Cosine similarity
Same meaning (should reuse)	0.881 – 1.000
Different meaning (must not reuse)	0.609 – 0.985
<code>return a and b</code> vs <code>return a or b</code>	0.985
<code>xs[:n]</code> vs <code>xs[:n-1]</code>	0.972
<code>a > b</code> vs <code>a >= b</code>	0.956

Table 3.2: Similarity ranges for code pairs.

The ranges overlap, and code that means the opposite scores near the top. The only threshold with no wrong reuse (0.99) misses four of five genuine reuses. For prose the groups separate cleanly (0.978 against 0.609), which is why the cache uses similarity for prose only and exact matching for code.

3.2.3 Cache: savings depend on repetition

The cache savings benchmark (`cache/benchmark/savings.py`) replays sessions of 20 requests through the gateway logic, varying how many requests repeat an earlier one exactly, and averages 25 seeded sessions per repeat rate (Table 3.3).

Requests that are repeats	Input tokens saved
0%	0.0%
20%	21.2%
40%	38.6%
50%	49.4%
60%	53.2%
80%	74.4%

Table 3.3: Cache savings against the share of repeated requests.

3.2.4 Cache: added latency

The latency benchmark (`cache/benchmark/latency.py`, median of 200 runs, with the embedding model installed) measures the time the cache adds to a request (Table 3.4). Exact matches, which include every code request, cost almost nothing. Only prose that is not an exact repeat pays for an embedding, and a miss pays twice, once in the lookup and once when the answer is stored.

Case	Added latency
Hit, exact repeat or code (no embedding)	about 0.003 ms
Hit, prose paraphrase (one embedding)	about 4.6 ms
Miss, prose (lookup and store, two embeddings)	about 8.4 ms
Cold start (one-time model load)	about 7.5 s

Table 3.4: Latency added by the cache.

3.2.5 Autopilot: secret detection

Against 4,000 randomly generated keys in the current OpenAI (`sk-proj-`) and Anthropic (`sk-ant-`) formats, the secret scan missed none. It does not flag the `notes_api` fixture, ordinary authentication code, or kebab-case text, so it does not switch off savings on normal requests.

3.2.6 End-to-end demonstration

The demo script (`python demo.py`) replays a scripted session of six requests through the real gateway with the mock provider. The gateway sent 7,309 of 19,824 tokens (63% fewer): 9,916 tokens were saved by cache hits and 2,599 by trimming a large file, and every request was answered. This figure depends on the script, which repeats a large request, so it illustrates the pipeline rather than predicting savings on other workloads.

3.3 Analysis

- **The trimmer beats text stripping and stays correct.** Stripping comments and docstrings, the simplest safe change, saves 20.5%. The trimmer saves 33.9% while keeping everything the task depends on. Removing indentation looks like a cheap win but produces broken Python.
- **Correctness costs savings, and that is the right trade.** Before the cross-file fix, the trimmer saved 53.8% on the same task, but only because it collapsed `validate_note`, which the fix needed. The realistic fixture is what exposed this; the toy project could not. Collapsing every body saves 57.0% for the same reason: it throws away the code being fixed.
- **Cache savings are set by the workload.** Savings track the share of repeated requests almost exactly, so a single “percent saved” figure says more about the chosen workload than about the cache. We therefore report the curve.
- **Whole-request caching rarely hits in multi-turn sessions.** An agent resends the whole conversation each turn, and the history grows, so the full request is almost never repeated exactly. In our test, the same question asked again on the third turn of a conversation did not hit. This limits the cache on real agent traffic.
- **The token budget decides whether the trimmer runs.** The trimmer only acts above the 8000-token budget. The `notes_api` request (3,277 tokens) is below it, so the measurements above set the budget explicitly. The autopilot asks for trimming above 1000 estimated tokens, so the two thresholds disagree; choosing one budget is part of the next stage.

Chapter 4

Conclusion

4.1 Summary of Work

The prototype runs end to end: an OpenAI-compatible gateway with a mock provider, a compiler-aware trimmer for Python that follows dependencies across files, a cache that reuses exact repeats and prose paraphrases while refusing similarity matches for code, and an autopilot that never caches secrets. It has 65 automated tests running in CI, design documentation on the project site, and a realistic fixture for measurement.

4.2 Key Findings

1. On a realistic request, the trimmer cuts 33.9% of tokens and the output is always valid code, against 20.5% for the best simple safe baseline.
2. A realistic fixture is essential: it exposed a dependency bug that had been inflating the trimmer’s savings.
3. No cosine threshold separates same-meaning from different-meaning code with a general-purpose embedding model, so code must use exact matching.
4. Cache savings equal the repetition rate of the workload, and whole-request matching rarely hits in multi-turn sessions.

4.3 Limitations

- Only Python is supported.
- The autopilot’s thresholds are fixed; tuning them from measured savings is not built yet, and its trim threshold is not yet aligned with the trimmer’s budget.
- Metrics (DFD process 6.0) are designed but not yet stored by the gateway.
- Calls to a class, such as `raise ConfigError(...)`, do not yet create call-graph edges, so the class’s constructor can be collapsed.
- The trimmer’s savings are measured on one fixture (four tasks) so far.

4.4 Future Work and Recommendations

- A reproducible ablation benchmark (raw, trimmer only, cache only, both) across several tasks and fixtures, and a principled choice of token budget.
- A second language (TypeScript) for the trimmer.
- Autopilot threshold tuning from measured savings, with one shared trim threshold.
- A metrics store and a savings report for developers.
- For multi-turn traffic, caching at the level of the conversation prefix rather than the whole request, and evaluating a code-aware embedding model.
- A benchmark across at least two coding agents through one endpoint.

4.5 Final Remarks

The prototype shows that a gateway which understands code structure can reduce what a coding agent sends without breaking it, and it gives us honest baselines to measure every later improvement against. The measurements also changed our plans: they showed where the easy percentage figures were misleading, and where the next real gains are.

Bibliography

- [1] Databricks, “Managing AI coding costs at scale,” Databricks Blog. <https://www.databricks.com/blog/managing-ai-coding-costs-scale>
- [2] Tree-sitter, an incremental parsing system. <https://tree-sitter.github.io/tree-sitter/>
- [3] OpenAI, *tiktoken*: a fast BPE tokeniser. <https://github.com/openai/tiktoken>
- [4] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” *EMNLP*, 2019.